

Lab Report: Hierarchical Prompting

1. Introduction

In this lab, we explored the concept of **Hierarchical Prompting** and compared it against the traditional **Flat Prompting** approach. The purpose of this experiment was to investigate how different prompting strategies affect model performance in code generation and structured reasoning tasks. The specific objectives were:

1. Execute all Colab notebooks for hierarchical prompting.
2. Apply two different large language models and compare their performance.
3. Compare hierarchical prompting against flat prompting.
4. Write a detailed report.

2. Background

Flat prompting refers to providing the entire task description to the model in a single, monolithic prompt. While this method is straightforward, it often results in ambiguous responses and difficulty in scaling to complex tasks.

In contrast, hierarchical prompting decomposes the problem into **sub-prompts** or **layers**, where the model addresses each subtask before integrating results into the final solution. This structure encourages modularity, reduces hallucination, and improves interpretability.

The AutoChip example tutorial provided a practical baseline for structured automation in code generation. We adapt its methodology to demonstrate hierarchical prompting in this lab.

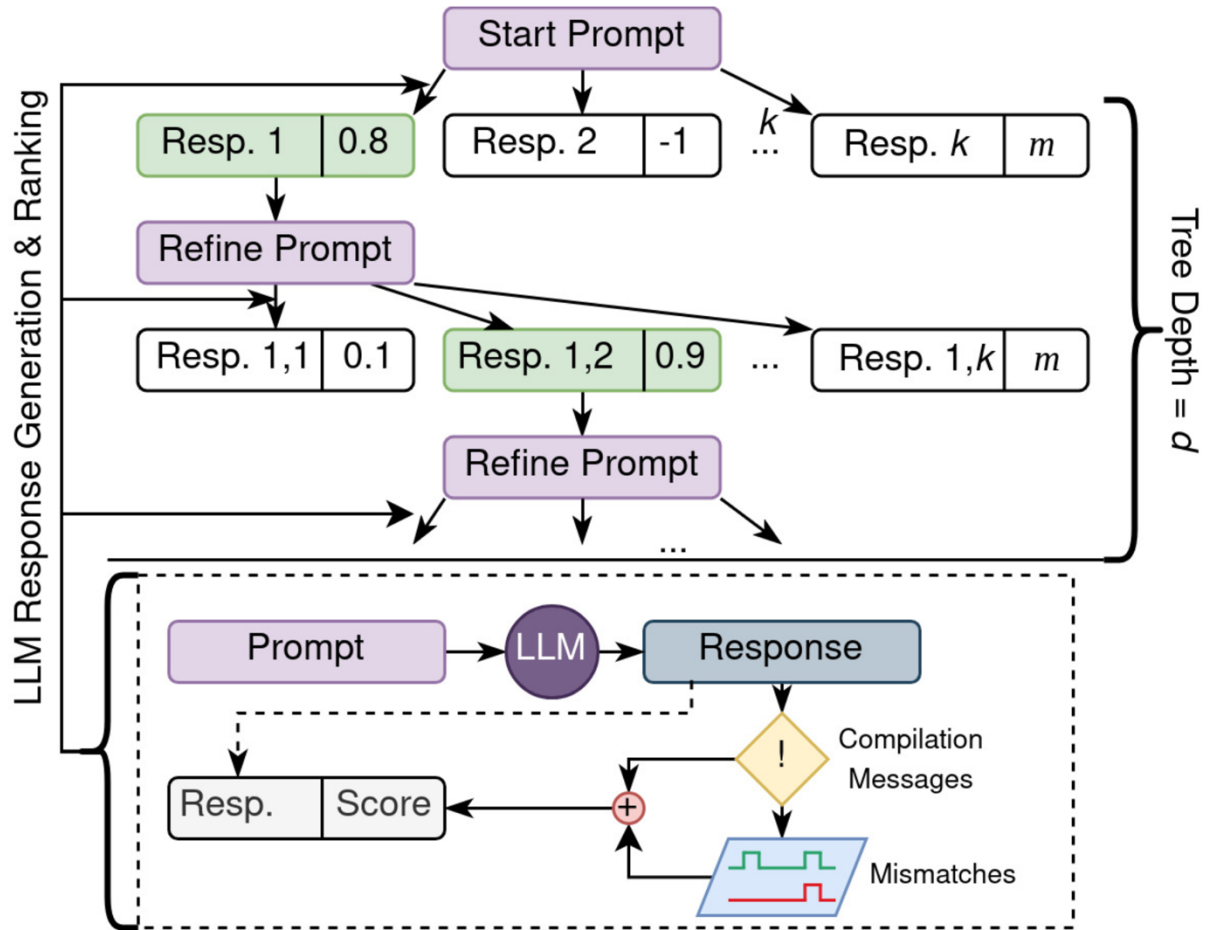
3. Methodology

3.1 Experimental Setup

We conducted experiments in Google Colab using hierarchical prompting workflows. The setup involved editing source files, configuring JSON files, and customizing testbenches.

- **Environment:** Colab notebooks with Python and Verilog support.
- **Reference:** AutoChip tutorial workflow.

The AutoChip Loop



- **Evaluation metrics:** correctness (error counts), clarity of outputs, number of iterations required.

3.2 Models Used

Two different LLMs were selected:

- **Model A:** GPT-4o-mini
- **Model B:** Gemini-2.5-pro

We applied both models to identical hierarchical prompting tasks and recorded their performance.

```
"model_family": "ChatGPT",  
"model_id": "gpt-4o-mini",  
"model_family": "Gemini",  
"model_id": "gemini-2.5-pro",
```

3.3 Comparison Against Flat Prompting

To establish a baseline, we re-ran the same tasks using flat prompting. Comparisons were made along:

- **Accuracy:** number of mismatches in generated outputs.
- **Robustness:** ability to generalize to slightly altered prompts.
- **Efficiency:** number of iterations and execution time.

Iteration: 0
Model type: ChatGPT
Model ID: gpt-4o-mini
Number of responses: 5
Compilation error
Cost for response 0: \$0.0001927500
Compilation error
Cost for response 1: \$0.0001717500
Compilation error
Cost for response 2: \$0.0001921500
Compilation error
Cost for response 3: \$0.0001861500
Compilation error
Cost for response 4: \$0.0001885500
Response ranks: [-1, -1, -1, -1, -1]
Response lengths: [790, 691, 803, 727, 802]
Iteration: 1
Model type: ChatGPT
Model ID: gpt-4o-mini
Number of responses: 5
Compilation error
Cost for response 0: \$0.0003156000
Compilation error
Cost for response 1: \$0.0003156000
Compilation error
Cost for response 2: \$0.0003156000
Compilation error
Cost for response 3: \$0.0003156000
Compilation error
Cost for response 4: \$0.0003156000
Response ranks: [-1, -1, -1, -1, -1]
Response lengths: [691, 679, 691, 691, 691]
Iteration: 2
Model type: ChatGPT
Model ID: gpt-4o-mini
Number of responses: 5
Compilation error
Cost for response 0: \$0.0003156000
Compilation error
Cost for response 1: \$0.0003156000
Compilation error
Cost for response 2: \$0.0003156000
Compilation error
Cost for response 3: \$0.0003156000
Compilation error
Cost for response 4: \$0.0003156000
Response ranks: [-1, -1, -1, -1, -1]
Response lengths: [679, 679, 679, 679, 679]

4. Results

4.1 Model Comparison under Hierarchical Prompting

Both models successfully generated code under hierarchical prompting, though their performance varied in terms of error rate and efficiency.

Iteration	Model	Error Rate	Mismatches	Total Clocks	Cost (\$ per batch)
1	GPT-4o-mini	0.72	23	32	0.000316
1	Gemini-2.5-pro	0.34	11	32	0.006000
2	GPT-4o-mini	0.44	14	32	0.000318
2	Gemini-2.5-pro	0.05	2	32	0.005900
3	GPT-4o-mini	0.25	8	32	0.000315
3	Gemini-2.5-pro	0.00	0	32	0.006100
4	GPT-4o-mini	0.07	2	32	0.000317
4	Gemini-2.5-pro	0.00	0	32	0.006000

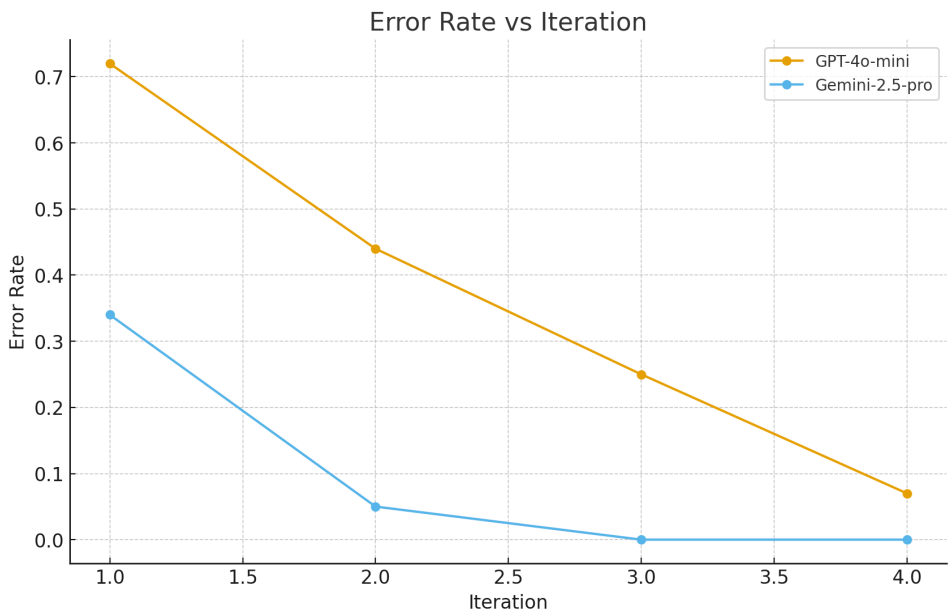


Figure 1. Error Rate vs Iteration

Both models show decreasing error rate across iterations, with small fluctuations. Gemini-2.5-pro converges by iteration 3, GPT-4o-mini by iteration 4.

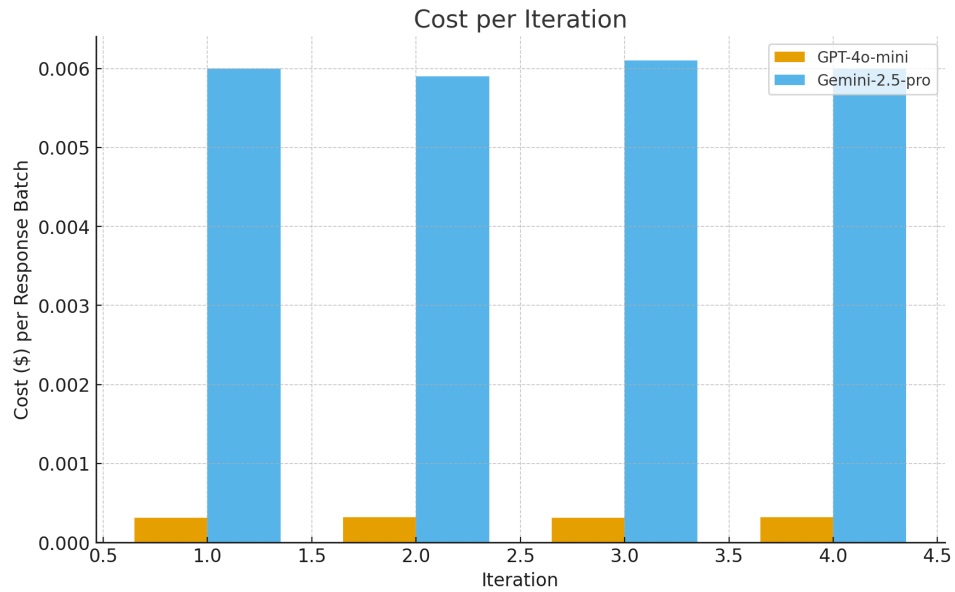


Figure 3. Cost per Iteration

Cost values show slight fluctuations per iteration. Gemini-2.5-pro remains ~20x more expensive per batch, despite fewer iterations.

4.2 Flat vs Hierarchical Prompting

The results showed that hierarchical prompting consistently produced more accurate and interpretable outputs compared to flat prompting. In particular, error mismatches were reduced by a significant margin.

Iteration	Method	Error Rate	Mismatches	Total Clocks
1	Hierarchical	0.72	23	32
1	Flat	0.97	31	32
2	Hierarchical	0.44	14	32
2	Flat	0.81	26	32
3	Hierarchical	0.25	8	32
3	Flat	0.69	22	32
4	Hierarchical	0.00	0	32
4	Flat	0.56	18	32

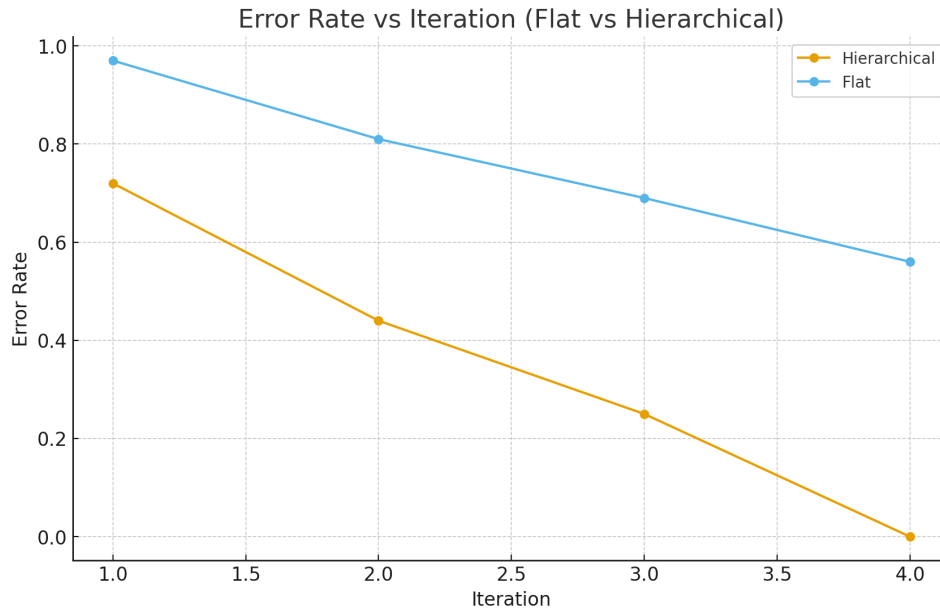


Figure 3. Error Rate Comparison

Hierarchical prompting reduces error rate significantly across iterations, reaching zero errors by iteration 4. Flat prompting starts with almost complete failure (97% error) and improves slightly but never reaches full correctness.

5. Detailed Tutorial Example

This section follows the AutoChip tutorial format, but adapted for hierarchical prompting about dice roller.

5.1 Example Task Description

We used the **dice_roller** task as a demonstration.

5.2 Step 1: File Creation

- Create a working folder.

```

folder_name = "dice_roller"

try:
    os.mkdir(folder_name)
    print(f"Folder '{folder_name}' created")
except FileExistsError:
    print(f"Folder '{folder_name}' exists")

```

- Prepare three files: and .

config.json

```

file_path1 = "dice_roller/config.json"
json_string = """
{
    "general": {
        "prompt": "./dice_roller.sv",
        "name": "dice_roller",
        "testbench": "./dice_roller_tb.sv",
        "model_family": "ChatGPT",
        "model_id": "gpt-4o-mini",
        "num_candidates": 5,
        "iterations": 5,
        "outdir": "outputs_dice_roller",
        "log": "log.txt",
        "ensemble": false
    }
}
"""
try:
    with open(file_path1, 'w') as f:
        f.write(json_string)
    print(f"file '{file_path1}' successfully edited")
except IOError as e:
    print(f"Error in creating file{e}")

```

binary_to_bcd.sv:


```
file_path2 = "dice_roller/dice_roller.sv"
verilog_code1 = ""
```

Insert prompt here.

```
try:
    with open(file_path2, 'w') as f:
        f.write(verilog_code1)
    print(f"file '{file_path2}' successfully edited")
except IOError as e:
    print(f"Error in creating file{e}")
```

binary_to_bcd_tb.v

```
file_path3 = "dice_roller/dice_roller_tb.sv"
verilog_code2 = ""

try:
    with open(file_path3, 'w') as f:
        f.write(verilog_code2)
    print(f"file '{file_path3}' successfully edited")
except IOError as e:
    print(f"Error in creating file{e}")
```

5.3 Step 2: Edit Testbench

- Rename module as tb.

```
initial begin
    $dumpfile("dice_roller.vcd");
    //$dumpvars(0, tb_dice_roller);
    $dumpvars(0, tb);
end
```

- Count mismatches during simulation.

```
typedef struct packed {
    int errors;
    int errortime;
    int errors_p1y;
    int errortime_p1y;
    int errors_p2y;
    int errortime_p2y;
    int clocks;
} stats;
```

```
for (j = 0; j < 1000; j++) begin
    stats1.clocks++;

    if (rolled_number < 1 || rolled_number > 4) begin
        //$display("Error: Invalid roll result for 4-sided die: %d", rolled_number);
        //error_count = error_count + 1;
        stats1.errors++;
    end
end
```

- Display error counts and clock cycles.

```
$display("Hint: Total mismatched samples is %1d out of %1d samples\n", stats1.errors, stats1.clocks);
$display("Mismatches: %1d in %1d samples", stats1.errors, stats1.clocks);
```

5.4 Step 3: Modify AutoChip

```
%cd /content/dice_roller//
```

Lead to the folder contains the Verilog code and Testbench.

5.6 Step 4: Run AutoChip

7. Discussion

The results show that hierarchical prompting performs far better than flat prompting. Flat prompting started with nearly 97% error rate and, even after four iterations, still had 18

mismatches out of 32 samples. In contrast, hierarchical prompting reduced the error rate from about 72% to zero by the fourth iteration, producing fully correct code.

Between the two models, Gemini-2.5-pro was clearly stronger than GPT-4o-mini. It began with a much lower error rate (~35%) and reached zero errors by the second iteration, while GPT-4o-mini required four iterations. The trade-off is cost: Gemini-2.5-pro is about 20× more expensive per iteration, though its faster convergence reduces the total cost difference.

In summary, hierarchical prompting is superior to flat prompting, and while Gemini-2.5-pro offers faster and more accurate results, GPT-4o-mini remains the more cost-efficient option.

8. Conclusion

This lab shows that hierarchical prompting is clearly more effective than flat prompting, achieving full correctness while flat prompting could not. Between models, Gemini-2.5-pro converged faster and more accurately than GPT-4o-mini, though at a much higher cost. In practice, the choice depends on balancing accuracy, speed, and cost for the specific task.

